

OP2

Generated by Doxygen 1.9.4

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Human Class Reference	7
4.2 Student1 Class Reference	8
4.2.1 Member Function Documentation	9
4.2.1.1 getPavarde()	10
4.2.1.2 getVardas()	10
5 File Documentation	11
5.1 header.h	11
5.2 human.hpp	11
5.3 student.hpp	12
Index	17

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Human	7
Student1	8

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Human		7
Student1		8

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

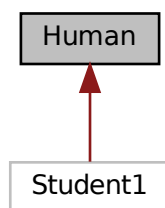
/home/xxmp/GitHub/OP2/program/ header.h	11
/home/xxmp/GitHub/OP2/program/ human.hpp	11
/home/xxmp/GitHub/OP2/program/ student.hpp	12

Chapter 4

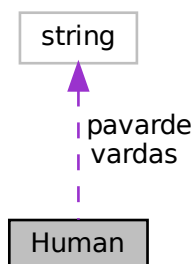
Class Documentation

4.1 Human Class Reference

Inheritance diagram for Human:



Collaboration diagram for Human:



Public Member Functions

- **Human** (const std::string &v, const std::string &p)
- virtual void **setVardas** (std::string vardas_)=0
- virtual void **setPavarde** (std::string pavarde_)=0
- virtual std::string **getVardas** () const =0
- virtual std::string **getPavarde** () const =0

Protected Attributes

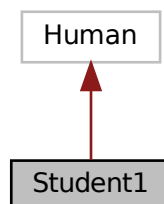
- std::string **vardas**
- std::string **pavarde**

The documentation for this class was generated from the following file:

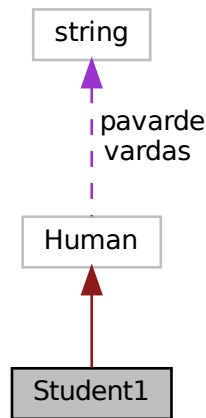
- /home/xxmp/GitHub/OP2/program/human.hpp

4.2 Student1 Class Reference

Inheritance diagram for Student1:



Collaboration diagram for Student1:



Public Member Functions

- **Student1** (string v="Default", string p="Default", int e=0, vector< int > m={0})
- **Student1** (const [Student1](#) &other)
- [Student1](#) & **operator=** (const [Student1](#) &other)
- **Student1** ([Student1](#) &&other) noexcept
- [Student1](#) & **operator=** ([Student1](#) &&other) noexcept
- bool **operator==** (const [Student1](#) &other) const
- void **setVardas** (string vardas_) override
- void **setPavarde** (string pavarde_) override
- void **setEgzaminas** (int egzaminas_)
- void **setMark** (vector< int > mark_)
- string [getVardas](#) () const override
- string [getPavarde](#) () const override
- int [getEgzaminas](#) () const
- vector< int > [getMark](#) () const
- float [getMedian](#) () const
- float [getResult](#) () const
- void **GenerateNames** ()

Friends

- std::ostream & **operator<<** (std::ostream &os, const [Student1](#) &stud)
- std::ifstream & **operator>>** (std::ifstream &in, [Student1](#) &stud)
- std::istream & **operator>>** (std::istream &is, [Student1](#) &stud)

4.2.1 Member Function Documentation

4.2.1.1 getPavarde()

```
string Student1::getPavarde ( ) const [inline], [override], [virtual]
```

Implements [Human](#).

4.2.1.2 getVardas()

```
string Student1::getVardas ( ) const [inline], [override], [virtual]
```

Implements [Human](#).

The documentation for this class was generated from the following files:

- /home/xxmp/GitHub/OP2/program/student.hpp
- /home/xxmp/GitHub/OP2/program/student.cpp

Chapter 5

File Documentation

5.1 header.h

```
1 #include <algorithm>
2 #include <cctype>
3 #include <cstdio>
4 #include <cstdlib>
5 #include <iomanip>
6 #include <iostream>
7 #include <random>
8 #include <sstream>
9 #include <string>
10 #include <fstream>
11 #include <vector>
12 #include <chrono>
13 #include <filesystem>
14 #include <ios>
15 #include <stdexcept>
16
17 using std::cout;
18 using std::cin;
19 using std::endl;
20 using std::setw;
21 using std::string;
22 using std::vector;
23 using std::ifstream;
24 using std::cerr;
25 using std::left;
26 using std::right;
27
28 bool isValid ( string number );
29
30 bool isValid ( int number );
31
32 int randomNumber (int a, int b);
33
34 void readFile(vector<string>& name);
35
36 string get_name(vector <string> name);
37
38 //void count_marks (vector<Student>& stud);
39
40 vector<string> listTxtFiles();
41
42 void generate_file (int number);
43
44
```

5.2 human.hpp

```
1 #include <string>
2
3 class Human {
4     protected:
5
6     std::string vardas;
7     std::string pavarde;
```

```

8
9     public:
10
11         // Default constructor
12         Human() : vardas(""), pavarde("") {}
13
14         // Parameterized constructor
15         Human(const std::string& v, const std::string& p) : vardas(v), pavarde(p) {}
16
17
18         // virtualus destruktorius
19         virtual ~Human() {
20             //std::cout << "Virtual destructor Human was called" << std::endl;
21         }
22
23         //virtualus setteriai
24         virtual void setVardas(std::string vardas_) = 0;
25
26         virtual void setPavarde(std::string pavarde_) = 0;
27
28         //virtualus getteriai
29         virtual std::string getVardas() const = 0;
30
31         virtual std::string getPavarde() const = 0;
32 };

```

5.3 student.hpp

```

1 #include <iomanip>
2 #include <ios>
3 #include <istream>
4 #include <ostream>
5 #include <fstream>
6 #include <sstream>
7 #include <string>
8 #include <vector>
9 #include <iostream>
10 #include <chrono>
11 #include <algorithm>
12 #include "../program/human.hpp"
13
14 using std::string;
15 using std::vector;
16
17
18 class Student1 : private Human {
19
20     private:
21         //string vardas;
22         //string pavarde;
23         int egzaminas;
24         vector<int> mark;
25     public:
26
27         // konstruktorius
28         Student1(string v = "Default", string p = "Default", int e = 0, vector<int> m = {0}) :
29 egzaminas(e), mark(m) {
30             //std::cout << "Iskviestas Konstruktorius studentui " << v << std::endl;
31             vardas = v;
32             pavarde = p;
33         }
34
35         // destruktorius
36         ~Student1() {
37             //std::cout << "Iskviestas Destruktorius studentui " << vardas << std::endl;
38             mark.clear();
39         }
40
41         // copy konstruktorius
42         Student1(const Student1 &other) : Student1(other.vardas, other.pavarde, other.egzaminas,
43 other.mark) { // kodel jei cia parasau : Student1 vardas = other.vardas....
44         Student1(const Student1 &other) { // kodel jei cia parasau : Student1 vardas = other.vardas....
45             vardas = other.vardas;
46             pavarde = other.pavarde;
47             egzaminas = other.egzaminas;
48             mark = other.mark;
49             //std::cout << "Iskviestas Copy Konstruktorius studentui " << vardas << std::endl;
50         }
51
52         // copy assignment
53         Student1& operator=(const Student1 &other) {
54             if (this != &other) { // Patikrinti ar ne self-assigninama
55                 vardas = other.vardas;

```



```

54         pavarde = other.pavarde;
55         egzaminas = other.egzaminas;
56         mark = other.mark;
57     }
58     else std::cout << "Self-assignment, skipping" <<std::endl;
59     //std::cout << "Iskviestas Copy Assignmentas studentui " << vardas <<std::endl;
60     return *this;
61 }
62
63 //move konstruktorius
64 Student1(Student1&& other) noexcept : Student1(other.vardas ,other.pavarde, other.egzaminas,
other.mark) {
65     //other = Student1();
66     //std::cout << "Iskviestas Move Konstruktorius studentui " << vardas << std::endl;
67     //}
68
69 Student1(Student1&& other) noexcept :
70 Human(std::move(other.vardas), std::move(other.pavarde)), // Move base class members
71     egzaminas(other.egzaminas),
72     mark(std::move(other.mark)) {
73     // Reset the moved-from object
74     other.vardas = "Default";
75     other.pavarde = "Default";
76     other.egzaminas = 0;
77     other.mark = {};
78     //std::cout << "Iskviestas Move Konstruktorius studentui " << vardas << std::endl;
79 }
80
81 // move assignment
82 /*
83 Student1& operator=(Student1&& other) noexcept {
84 if (this != &other) { // Patikrinti ar ne self-moving
85 vardas = other.vardas;
86 pavarde = other.pavarde;
87 egzaminas = other.egzaminas;
88 mark = other.mark;
89
90 other = Student1();
91 }
92 else std::cout <<"Self-moving, skipping"<<std::endl;
93 //std::cout<<"Iskviestas Move Assignmentas studentui "<<vardas<<std::endl;
94 return *this;
95 }
96 */
97
98 Student1& operator=(Student1&& other) noexcept {
99     if (this != &other) { // Avoid self-move
100         // Move base class members
101         Human::operator=(std::move(other));
102
103         // Move member variables
104         egzaminas = other.egzaminas;
105         mark = std::move(other.mark);
106
107         // Reset the moved-from object
108         other.vardas = "Default";
109         other.pavarde = "Default";
110         other.egzaminas = 0;
111         other.mark = {};
112     }
113     //std::cout<<"Iskviestas Move Assignmentas studentui "<<vardas<<std::endl;
114     return *this;
115 }
116
117 // Equality operator
118 bool operator==(const Student1& other)const {
119     return (vardas == other.vardas &&
120         pavarde == other.pavarde &&
121         egzaminas == other.egzaminas &&
122         mark == other.mark);
123 }
124
125 // setteriai
126 void setVardas (string vardas_) override;
127
128 void setPavarde(string pavarde_) override;
129
130 void setEgzaminas(int egzaminas_);
131
132 void setMark(vector<int> mark_);
133
134 // getteriai
135
136 string getVardas()const override {
137     return vardas;

```

```

140     }
141
142     string getPavarde()const override {
143         return pavarde;
144     }
145
146     int getEgzaminas()const {
147         return egzaminas;
148     }
149
150     vector<int> getMark()const {
151         return mark;
152     }
153
154     friend std::ostream& operator<<(std::ostream& os, const Student1& stud) {
155         int g = 20;
156         os << std::setw(g) << std::left << stud.getVardas()
157         << std::setw(g) << std::left << stud.getPavarde()
158         << std::setw(g) << std::left << std::setprecision(3)
159         << stud.getResult() << std::setw(g) << std::left
160         << std::setprecision(3) << stud.getMedian() << std::endl;
161         return os;
162     }
163
164     // ifstream dar padaryt
165     friend std::ifstream& operator>>(std::ifstream& in, Student1& stud) {
166         std::string temp;
167         std::getline(in, temp); // Read the entire line
168         //std::cout << "nuskaitoma eilute \n" << std::endl;
169
170         if (temp.empty()) return in; // Skip empty lines
171
172         string random;
173         std::istringstream stream(temp); // Convert the line into a string stream
174         //std::cout << "konvertuojama i string stream \n" << std::endl;
175
176         stream >> stud.vardas >> stud.pavarde;
177
178         //std::cout << "ivesta vardas ir pavarde" << std::endl;
179         //std::cout << stud.getPavarde() << " " << stud.getPavarde() << "\n";
180
181         int marks;
182         stud.mark.clear(); // Clear marks vector
183         while (stream >> marks) {
184             stud.mark.push_back(marks);
185             //std::cout << "ivestas pazymys \n" << std::endl;
186         }
187
188         if (!stud.mark.empty()) {
189             stud.egzaminas = stud.mark.back();
190             stud.mark.pop_back();
191         } else {
192             stud.egzaminas = 0; // Default value if no marks are provided
193         }
194         //std::cout << "ivestas egzaminas \n" << std::endl;
195
196         return in;
197     }
198
199     }
200 }
201
202 friend std::istream& operator>>(std::istream& is, Student1& stud) {
203     std::string egzaminas, mark_temp;
204     vector<int> mark {};
205
206     std::cout << "Enter name: ";
207     is >> stud.vardas;
208
209     std::cout << "Enter surname: ";
210     is >> stud.pavarde;
211
212     while (true) {
213         std::cout << "Enter exam mark (0-10): ";
214         std::cin >> egzaminas;
215         if (std::stoi(egzaminas) >= 0 && std::stoi(egzaminas) <= 10) {
216             stud.egzaminas = std::stoi(egzaminas);
217             break;
218         }
219     }
220
221     while (true) {
222         std::cout << "Enter a mark (or 'q' to quit): ";
223         std::cin >> mark_temp;
224
225         if (mark_temp == "q") {
226             stud.mark = mark;

```

```
227         mark.clear();
228         system("clear");
229         break;
230     }
231
232     if (std::stoi(mark_temp) >= 0 && std::stoi(mark_temp) <= 10) {
233         mark.push_back(std::stoi(mark_temp));
234     }
235 }
236
237     return is;
238 }
239
240     float getMedian() const;
241
242     float getResult() const;
243
244     void GenerateNames();
245
246 };
247
248 void print(vector<Student1> &stud);
249
250 void import_file (vector<Student1> &stud, string filename);
251
252 void sort_students (vector<Student1>& stud);
253
254 void choose_printing_method (vector<Student1>& stud);
255
256 void print_marks (vector<Student1> stud);
257
258 void write_marks (vector<Student1> stud, string name);
259
260 void divide_file1 (vector<Student1>& stud,vector<Student1>& kietiakai,vector<Student1>& nuskriaustukai,
    string filename);
261
262 void divide_file2 (vector<Student1>& stud,vector<Student1>& nuskriaustukai, string filename);
263
264 void divide_file3 (vector<Student1>& stud,vector<Student1>& nuskriaustukai, string filename);
265
266 void pagrindinis_divide (vector<Student1>& stud,vector<Student1>& kietiakai,vector<Student1>&
    nuskriaustukai, int num);
267
268 void pagrindinis_divide_choice(vector<Student1>& stud);
269
270 void sort_file (vector<Student1>& stud, string name);
271
272 void rule_of_five();
273
274 void print_metrics (string filename, float data, int num, int strategija);
275
276 int lineCount (string filename);
277
278 string extractNumbers(const std::string& str);
```


Index

[/home/xxmp/GitHub/OP2/program/header.h](#), [11](#)
[/home/xxmp/GitHub/OP2/program/human.hpp](#), [11](#)
[/home/xxmp/GitHub/OP2/program/student.hpp](#), [12](#)

getPavarde
 Student1, [9](#)
getVardas
 Student1, [10](#)

Human, [7](#)

Student1, [8](#)
 getPavarde, [9](#)
 getVardas, [10](#)